CSCI 311 S25 City Connections Project Team 4

Dong Hyun Roh (dhr014), Joshual Lee (jjl032), Colin Soule (cjs056)

April 2025

Algorithmic Structure and Data Structures Used

In this project, we implemented **Prim's Algorithm** and **Kruskal's Algorithm** to compute the Minimum Spanning Tree (MST) of an undirected, weighted graph representing city connections. The graph is read from an input file and represented internally as an adjacency list.

Data Structures

- **Adjacency List**: The graph is represented using a `defaultdict(list)` from Python's `collections` module. Each node maps to a list of tuples representing edges in the format `(edge length, start node, end node, edge ID)`.
- **Disjoint Set**: The basis of Kruskal's Algorithm. Creates a list of length N, where each index represents a unique element and each value represents the element's parent set. These sets are disjoint, so every element is found in exactly one set.
- **Priority Queue (Min-Heap)**: A custom `MinHeap` class is used to select the minimum-weight edge. It maintains a binary heap using a list and supports `push()` and `pop()` operations with $O(\log n)$ complexity, ordering edges by weight.
- **Visited Set**: A standard Python `set()` tracks which nodes have already been added to the MST to avoid cycles.
- **Edge Set**: A set is used to store the unique edge IDs that form part of the final MST.

Asymptotic Runtime Analysis

Prim's Algorithm Description

- **Time Complexity (Worst Case)**:

$$O((V + E) \log V)$$

In the worst-case scenario, the graph is densely connected, and each edge added to the MST causes heap insertions for adjacent nodes. Each such insertion or removal in the min-heap takes $O(\log V)$ time. Thus, for all E edges and V nodes, the total time complexity becomes $O((V + E) \log V)$.

- **Space Complexity**:

$$O(|V| + |E|)$$

for storing the adjacency list, heap, visited set, and MST edge list.

Kruskal's Algorithm Description

- **Time Complexity (Worst Case)**:

$$O(|E| \log |V|)$$

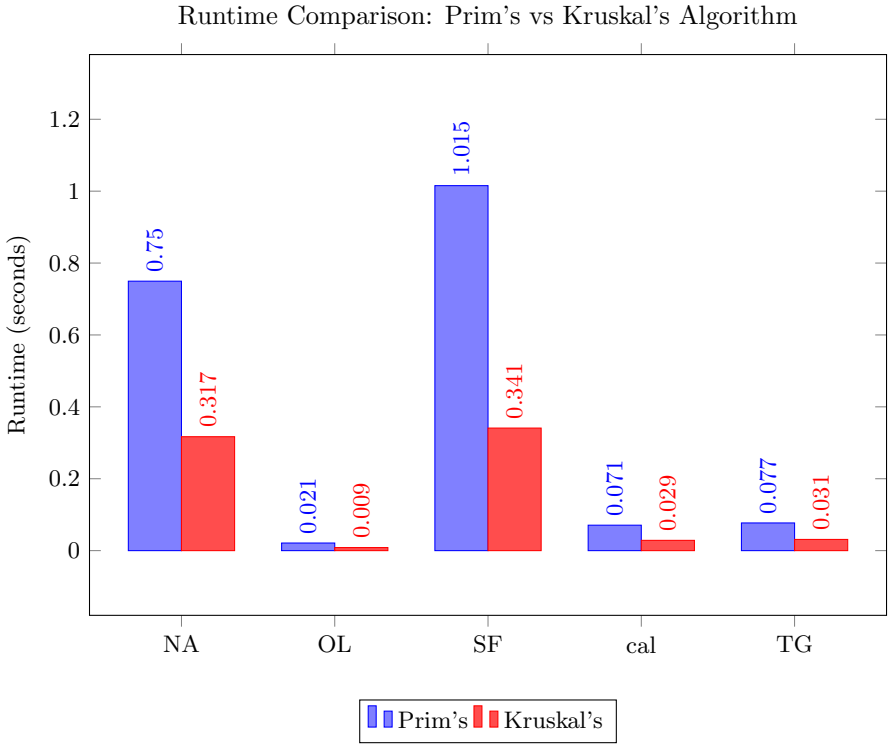
In the worst-case scenario, we have to check every edge in the graph to see if it would create a cycle. Thus, we have a for loop with $|E|$ iterations. However, at worst, cycle checking with a disjoint set takes $O(log|V|)$. This is because when we add an edge, we use the disjoint set union operator, which at worst points half of the items to a new head for its final union. all worst case unions before that cost half as much, giving us $\log |V|$ for our worst case scenario. When we do not add an edge, it costs $O(1)$ instead since we only use the disjoint set find, which goes to the index and returns the value there. Thus, the total time complexity is $O(|E| \log |V|)$.

- **Space Complexity**:

$$O(|V| + |E|)$$

In implementation, we use the adjacency list to calculate the number of nodes, the raw file to make a sorted edge list, and we create a disjoint set of size V.

Algorithmic Comparison



The graph above shows the measured runtime performance of Prim's and Kruskal's algorithms across five different datasets. As expected, Kruskal's algorithm consistently outperformed Prim's in every case. This is consistent with theoretical expectations, particularly given our implementation.

Prim's algorithm, implemented using a custom binary heap (MinHeap), requires multiple priority queue operations for each edge explored. While its theoretical complexity is $O((V + E) \log V)$, the overhead of maintaining the heap and inserting many edges can be costly in practice—especially for dense graphs.

In contrast, Kruskal's algorithm sorts all edges initially ($O(E \log V)$) and uses an efficient Union-Find data structure to construct the MST with minimal additional operations. This gives Kruskal's a performance edge in our measurements, particularly on larger datasets like SF and NA.

Overall, the experimental results validate our theoretical expectations: Kruskal's algorithm is more efficient for the graphs tested, both in runtime complexity and actual performance.